DODVins: A Data-Oriented MSCKF Pipeline for Predictable Visual-Inertial Odometry

Shafwan Safi, Nikhil Vashisht, Pranjal Upadhyay
Indian Institute of Science Education and Research (IISER) Bhopal

***Abstract*—We present DODVins, a data-oriented C++ reimplementation of the multi-state constraint Kalman filter (MSCKF) visual-inertial odometry (VIO) pipeline used by OpenVINS. The estimator's model is unchanged; what differs is the implementation: data-oriented memory layout — fixed-capacity per-frame state, no virtual dispatch, a feature database that separates search keys from payloads, allocation-free propagation — plus a small set of individually measured solver restructures, reported separately. Benchmarked on ten public sequences through the identical transport and toolchain container, DODVins matches OpenVINS's trajectory accuracy (better on 4 of 10, mean ATE 0.156 m vs. 0.158 m) while running 1.76–2.10× faster end-to-end (with a 99th-percentile frame-latency advantage that exceeds its mean advantage, the tail being where the layout wins) and a bounded 184–199 MB working set against OpenVINS's variable 98–179 MB — an intentional trade of lower average utilization for a bounded working set with allocation-free propagation. The design transfers to an NVIDIA Jetson Orin Nano (faster on all ten sequences, narrower margin), and the pipeline converges untuned on a third dataset family, TUM VI. We report each optimization's individually measured effect, two benchmarking confounds (toolchain, transport) that invalidate casually quoted ATEs, a cross-platform defect the embedded deployment caught, and what remains unvalidated. This is an implementation and measurement study: every filter equation is OpenVINS's; what differs is memory layout, and the paper reports what that change costs and buys, measured, not assumed.**

***Index Terms*—visual-inertial odometry, MSCKF, data-oriented design, embedded robotics, cache-efficient state estimation**

I. Introduction

Flying robots, AR/VR headsets, and embedded rovers share a hard constraint: state estimation must run per frame, on hardware that also has to drive control and navigation, within a power budget. For a team deploying a visual-inertial odometry (VIO) pipeline on such hardware, the estimator's mathematics is only half the problem; the other half is what its implementation costs on a per-frame hot path, in wall-clock, in the tail of the frame-latency distribution, and in memory. It is not clear from existing results in the literature what those costs actually are for the MSCKF [2], the filtering approach behind a large number of deployed VIO systems: existing MSCKF implementations are tuned for extensibility on research workstations, and no published measurement isolates how much of a production MSCKF pipeline's per-frame cost is the algorithm and how much is the design pattern of its implementation.

OpenVINS [1] is a widely used, carefully engineered open-source MSCKF implementation and is the accuracy and design reference for this work. Its C++ implementation follows conventional object-oriented design: polymorphic state variable types, `std::vector` containers, and heap-allocated per-feature and per-measurement objects. The point of this description is not to criticize OpenVINS's engineering but to measure what that design pattern costs on a per-frame hot path, independent of the estimator's mathematics. Data-oriented design (DOD) [6] argues that on such paths the layout of data in memory, not the algorithm expressed over it, is usually the dominant cost. We test that argument on a production-shaped VIO pipeline rather than a microbenchmark: the same MSCKF pipeline in C++ under a small set of house rules — no per-frame heap allocation, no virtual dispatch, all state in fixed-capacity tables — and measured against the original on identical data through an identical transport.

The result is not a strict win. Reporting it as one would misrepresent the data: DODVins is faster by a wide, consistent margin, and it is accuracy-competitive but not accuracy-superior. Section V presents both findings without rounding the second one away, and Section IX states plainly what has not yet been validated, including the parts of the embedded comparison that remain unmeasured (thermals, frequency state, wall-clock).

Contributions.

- An implementation redesign of the OpenVINS MSCKF pipeline — memory layout plus numerically equivalent solver restructures — in which every accepted optimization leaves DODVins's trajectories bit-identical to its own pre-change output across ten sequences, with per-optimization, independently measured speedups (Table V).
- A same-transport benchmarking methodology (identical bag, identical transport, both estimators as ROS nodes in the identical toolchain container) plus a measured decomposition of the two environmental confounds: transport choice moves the ATE by 0–5.2% across the EuRoC MH sequences, while toolchain choice moves it by up to 23% on identical code and data — an ATE quoted without both cannot be reproduced.
- A relative-pose-error decomposition showing that an aggregate ATE comparison can hide a steady-state accuracy advantage behind an initialization-timing artifact, and a worked example (Section V-F) where exactly this happens.
- A reported, not silently avoided, cross-platform numerical defect (Section VI): a companion-matrix eigenvalue convergence gate tuned implicitly for x86-SSE2 round-

ing that rejected a correct solution under ARM-NEON rounding, found via a Jetson Orin Nano deployment, root-caused, and fixed with the fix verified not to move any of the ten trajectories on x86.
- An embedded validation of the same comparison on the Jetson Orin Nano Super: both estimators, all ten sequences, per-frame latency and peak RSS under the identical toolchain container as x86 (Section VI), including the finding that the desktop tail advantage does *not* grow on the small core.

II. Related Work

OpenVINS [1] is the reference implementation and accuracy baseline throughout this paper; every comparison in Section V is against an unmodified, independently rebuilt copy of it, run through the same transport as DODVins under test.

SchurVINS [3] restructures the MSCKF update to exploit the block-diagonal structure of the per-measurement Hessian, reducing an update from $O(m^3)$ to linear in the number of features by eliminating the landmark block once per feature via a Schur complement before the pose update. We reimplemented SchurVINS's update in the same data-oriented layout (fixed-capacity buffers, no per-frame allocation) as an alternate MSCKF update path, gated behind a runtime flag and *off by default*; every benchmark number in this paper uses the primary MSCKF/SLAM update path, not the SchurVINS path.

sqrtVINS [4] contributes two ideas this pipeline uses directly and attributes precisely: a square-root-form EKF update that factors rather than inverts the innovation covariance (Section III-C), and a feature-less dynamic initializer (Section III-D) that recovers velocity and gravity from bearing-only epipolar geometry without ever triangulating a 3D point, which this pipeline uses as its primary EuRoC initialization path.

The hovering classifier follows Kottas, Wu, and Roumeliotis [5]: a rotation-compensated bearing-vector residual between consecutive clones, with hysteresis, used here to switch clone marginalization between FIFO (generic motion) and LIFO (hovering) so a stationary interval does not squeeze a genuine-motion baseline out of the sliding window.

III. System Design

A. The house rules

DODVins follows a small, fixed set of constraints applied uniformly to every file on the per-frame path:

1) **No variable-sized per-frame objects.** No `new` for features, measurements, or state variables, no `std::vector::push_back` on the frame path, no `std::string`, no `std::function`. All per-frame state lives in fixed-capacity structs or a bump-pointer arena reset at a known frame boundary. The dense linear algebra of the update step uses a bounded set of per-frame workspaces (measured in Section V); the propagation path allocates nothing, which a test asserts.
2) **All memory bounded at initialization.** Every array carries a compile-time capacity constant; `init_*()` functions validate the configured maximum against it and abort loudly rather than corrupt memory silently at the first overflow.
3) **No virtual dispatch, no inheritance, no RTTI on the hot path.** Behavior that varies by case varies by which fixed-layout table a row belongs to and which transform runs over that table, decided once per frame outside the inner loop, not per-row inside it.
4) **Existence-based processing.** Table membership is the boolean. A feature eligible for triangulation is one already present in a pre-filtered pointer array built once upstream; the triangulation loop itself carries no per-row conditional.
5) **Transforms are pure over their row.** A transform reads its inputs and writes its outputs once; it does not read back what it just wrote, so aliasing analysis and the accuracy argument for a given change both stay tractable.

These restate data-oriented design as documented by Fabian [6]; the contribution is applying them uniformly to a full MSCKF pipeline and measuring the result.

B. The feature database

The most consequential single layout decision is in the feature database. OpenVINS's `FeatureDatabase` is a hash map from feature ID to a heap-allocated feature object; DODVins uses a fixed array of 2048 feature slots (measured peak occupancy across all ten benchmark sequences: 250, on MH_04) with search order and payload storage *physically separated*: an `order[]` array of 4-byte slot indices is kept sorted by feature ID, while the 2400-byte feature payloads themselves never move once written. Before this change, the database kept payloads physically sorted by ID, so every newly observed feature required a `memmove` of the array tail to preserve order — measured at 9,556 feature copies per frame, 22.9 MB/s of pure `memcpy`, to maintain an ordering over what is structurally a 4-byte key. Separating key from payload turns that insertion cost into a shift of 4-byte integers instead of 2400-byte structs, and keeps feature payload addresses stable across compaction.

C. Square-root-form EKF update

The EKF update is restructured following sqrtVINS's [4] central principle: never invert what can be factored. With innovation covariance $S = H_{\text{stack}} P H_{\text{stack}}^\top + R$, the conventional form computes S^{-1} explicitly (an m^3 solve against the identity) and then $K = M_a S^{-1}$ as a separate dense product. Writing $S = LL^\top$ (Cholesky) and $G := M_a L^{-\top}$:

$$KM_a^\top = M_a S^{-1} M_a^\top = (M_a L^{-\top})(M_a L^{-\top})^\top = GG^\top \quad (1)$$

so the explicit inverse and the separate K product both disappear, and the covariance update becomes a symmetric rank-k update on the upper triangle of P. This is faster, and it also eliminates a real correctness defect present in an earlier, dense form of this update. $KM_a^\top$ is symmetric in exact arithmetic but not in floating point, and collapsing the update to

the dense form `Cov -= K * M_a.transpose()` allows that asymmetry to drive covariance diagonals negative within a few hundred frames — measured to abort the filter on EuRoC MH_01 at approximately t=16 s. The rank-k form is symmetric by construction and does not exhibit this failure.

For scale: the speed contribution is real but small — 0.29 ms per frame across the two affected stages (Table V, row 3), roughly 8% of the total per-frame reduction from all accepted changes. The square-root form's primary contribution is the correctness property above; the speed difference is secondary.

D. Feature-less dynamic initialization

The standard MSCKF dynamic initializer solves jointly for feature depths, velocity, and gravity from a short window of vision and IMU preintegration. Over a short, low-parallax window this system is close to rank-deficient in the feature block: measured against Leica ground truth on EuRoC MH_02, it recovered a velocity of 0.162 m/s where ground truth was 0.029 m/s, and the resulting run diverged, while every quality metric available to the filter at initialization time — residual, conditioning, recovered gravity direction — looked healthy. Following sqrtVINS Sec. V-A [4], the pipeline's primary EuRoC initializer instead recovers velocity and gravity *without* ever estimating a 3D point: bearings from a feature observed in two frames, rotated into a common reference frame, span an epipolar plane whose normal is perpendicular to the relative translation direction; stacking this constraint over many features and combining it with IMU preintegration (which supplies the translation *with* scale) yields a linear system in velocity and gravity alone. This initializer fires successfully at the minimum admissible window (3 feature pairs, 3 time instants) on all eight EuRoC sequences benchmarked in this paper.

IV. Experimental Setup

A. Hardware

All x86 measurements were taken on an AMD Ryzen 9 7950X (16 cores / 32 threads), 61 GB RAM, running the estimator and OpenVINS as ROS 1 Noetic nodes inside an identical Docker container on both sides of every comparison.

B. Same-transport methodology

Every accuracy and wall-clock comparison in this paper runs both estimators as ROS 1 nodes consuming the identical rosbag through the identical transport, rather than comparing DODVins's native dataset reader against the reference implementation's ROS node. We measured what transport choice alone is worth: running DODVins on all five EuRoC MH sequences through the dataset's raw ASL folder format versus through a played rosbag, inside the identical toolchain container, moves the ATE by 0–5.2% per sequence (e.g. MH_01 0.1293 m through both transports; MH_02 0.2080 m ASL versus 0.1978 m bag). A previously reported approximately 15% transport effect was a misattribution: its ASL-side figure (0.1131 m) could not be reproduced at any commit we tested (Section IV-C), and the large environmental effect on accuracy is the *toolchain*, not the transport. The V1 sequences are quoted from the ASL transport only, because their bag ground-truth topic requires a conversion the bag runner does not implement; MH and KAIST bag transport carries ground truth directly.

TABLE I
Identical code and data, two toolchains (ATE RMSE, m; Δ relative to the container figure). Most sequences agree within a few percent; MH_02 differs by 23%.

Sequence	container	host	Δ
MH_01	0.1293	0.1282	1%
MH_02	0.2080	0.1595	23%
MH_03	0.2343	0.2234	5%
MH_04	0.4267	0.4416	3%
MH_05	0.3285	0.3115	5%
V1_01	0.0545	0.0499	8%
V1_02	0.0482	0.0553	15%
V1_03	0.0550	0.0564	3%

C. Toolchain confound

Transport is not the only environmental variable an ATE figure silently carries. Building the identical estimator code from the identical commit against identical data, differing only in the toolchain (OpenCV 4.2.0 with g++ 9.4.0 in a pinned Docker container versus OpenCV 4.5.4 with g++ 11.4.0 on the host), reproduces most EuRoC sequences within about 1% but differs by 23% on MH_02 and 8% on V1_01 (Table I). The frontend is OpenCV's KLT tracker, so the library version is part of the reported result, and the divergence concentrates exactly where reviewers spot-check. We quote an environment with every ATE in this paper; an ATE quoted without its transport and toolchain cannot be reproduced.

D. Datasets

Ten public sequences: EuRoC MH_01–MH_05 (machine hall) and V1_01–V1_03 (vicon room), and two KAIST Complex Urban sequences (circle, infinity), covering hand-held/UAV indoor flight and ground-vehicle outdoor driving. MH_* ground truth is the bag's `/leica/position` topic; V1_* ships vicon pose on a differently named topic that the default extraction does not handle, so V1_* is scored against the dataset's ASL ground-truth stream instead.

E. Wall-clock methodology

Wall-clock comparisons use `hyperfine` 1.18.0, `-i` (ignore non-zero exit — OpenVINS's ROS node throws a benign `class_loader::LibraryUnloadException` at shutdown, after all estimation work and file output are complete), one warmup run, five measured runs.

F. Allocation methodology

Allocation profiling uses `valgrind`'s DHAT tool over 12 seconds of EuRoC MH_01 played through the ASL runner, ranked both by allocation count and by total bytes, since the two rankings identify different defects: a site making many small allocations and a site making few very large ones are both invisible to the other's ranking.

TABLE II
ABSOLUTE TRAJECTORY ERROR (ATE RMSE, METRES), SAME ROSBAG TRANSPORT FOR BOTH ESTIMATORS. BOTH COLUMNS RE-MEASURED ON 2026-08-30 FROM THE CURRENT TREE INSIDE THE PINNED TOOLCHAIN CONTAINER; EVERY FIGURE IN THIS TABLE IS REPRODUCIBLE FROM THE RELEASED ARTIFACT.

Sequence	DODVins	OpenVINS	Ratio
MH_01_easy	0.1293	**0.1180**	1.10
MH_02_easy	0.1978	**0.1616**	1.22
MH_03_medium	**0.2342**	0.2486	0.94
MH_04_difficult	0.4267	**0.4110**	1.04
MH_05_difficult	**0.3163**	0.3195	0.99
V1_01_easy	0.0751	**0.0634**	1.18
V1_02_medium	0.0589	**0.0573**	1.03
V1_03_difficult	**0.0550**	0.0595	0.92
KAIST circle	0.0374	**0.0305**	1.23
KAIST infinity	**0.0261**	0.0284	0.92
Mean	0.1557	0.1583	—
Geometric mean of ratio			1.056

TABLE III
END-TO-END WALL-CLOCK AND TOTAL CPU TIME (USER+SYSTEM), HYPERFINE, 5 RUNS, SAME ROSBAG TRANSPORT. THE CPU COLUMN IS TOTAL PROCESSING WORK ACROSS ALL CORES; THE WALL/SPEEDUP RATIO IS 1.76–2.10×.

	DODVins		OpenVINS	
Sequence	Wall (s)	CPU (s)	Wall (s)	CPU (s)
MH_01	14.034 ± 0.072	29.5	29.450 ± 0.166	44.9
MH_04	7.654 ± 0.053	16.3	15.666 ± 0.044	24.3
circle	15.689 ± 0.027	32.5	27.628 ± 0.142	42.8

TABLE IV
PER-FRAME PIPELINE LATENCY, X86 (TRACKING + ESTIMATION, EXCLUDING BAG I/O; MS; IDENTICAL ROSBAG TRANSPORT AND TOOLCHAIN CONTAINER FOR BOTH ESTIMATORS).

	DODVins		OpenVINS	
Sequence	p50	p99	p50	p99
MH_01	3.6	6.7	6.6	13.6
MH_02	3.6	6.9	6.5	13.4
MH_03	3.9	7.3	6.8	14.6
MH_04	3.6	7.2	6.2	13.9
MH_05	3.7	7.2	6.5	14.8
V1_01	4.4	7.4	7.3	15.4
V1_02	4.3	7.6	7.0	14.0
V1_03	3.9	8.0	6.4	13.3
circle	3.4	5.9	6.2	10.3
infinite	3.7	6.1	6.4	10.0

V. RESULTS

A. Accuracy

Table II shows DODVins better on 4 of 10 sequences, worse on 6, mean ATE 0.1557 m against OpenVINS's 0.1583 m, geometric-mean per-sequence ratio 1.056. We report this as *accuracy parity*, not an accuracy advantage. Every figure in the table was re-measured for this paper from the current code in the pinned container (both estimators), with OpenVINS's ground-truth-adjacent outputs produced by its own state recorder; the V1 rows are evaluated against the dataset's ASL ground-truth stream for both estimators because the bag transport does not carry that topic in a form the extraction handles.

B. Wall-clock throughput

Table III shows 1.76–2.10× end-to-end speedup with identical trajectories, at points where the accuracy and speed measurements were taken from the same build and the same run. The CPU column shows the same ordering in total processing work: DODVins spends 24–34% less processor time per run than OpenVINS, which is the quantity a power- or duty-cycled platform actually pays for.

C. Allocation profile

Over 12 seconds of EuRoC MH_01, DHAT reports 777,007 heap blocks and 1.89 GB total allocated (down from an earlier 1,322,658-block baseline after eliminating three sites responsible for nearly all of the DODVins-attributable churn). Attributing every allocation site by its call stack: 59% of blocks are OpenCV's KLT tracker internals, 15% are one-time library startup, 3% dataset I/O, and 19% belong to DODVins's estimator path — and those 19% are the honest caveat to the allocation-free claim: the update step allocates a bounded set of dense Eigen workspaces every frame, measured at 137 blocks and 3.8 MB per frame in `EKFUpdate`. What *is* allocation-free is the propagation path (asserted by a unit test) and the per-frame state itself: no per-feature or per-measurement objects are ever heap-allocated, and the working set does not grow with sequence length or feature count. The prediction-free alternative — per-feature heap objects, as in the reference design — allocates in proportion to scene content instead.

D. Per-frame latency and predictability

Mean throughput is the metric a real-time reviewer trusts least, because it hides exactly the tail a real-time system cares about. Every run in this paper therefore records per-frame tracking-plus-estimation latency (excluding bag I/O) for both estimators. Table IV reports p50/p99/max on x86 (full per-sequence distributions in the released artifact).

The 99th-percentile ratio (1.64–2.05×) exceeds the median ratio (1.63–1.85×) on 8 of 10 sequences: the tail is where the data-oriented layout pulls further ahead, not behind. Worst observed frames are 8.3–18.4 ms for DODVins against 19.8–54.8 ms for OpenVINS, whose single worst frame (KAIST circle, 54.8 ms) exceeds the 16 Hz frame budget of that dataset. This is the paper's strongest single latency number, and it is measured, not argued.

E. Per-optimization ablation

Table V reports each change's independently measured effect, each verified against the full ten-sequence ATE gate before being accepted. Two plausible-sounding hypotheses were profiled and found wrong: capping DODVins at OpenVINS's own 40-feature update cap changed the MSCKF stage by only 3% and cost accuracy on 6 of 8 sequences (reverted), and the

TABLE V

INDIVIDUALLY MEASURED OPTIMIZATIONS, CLASSIFIED BY KIND. EVERY ROW: DODVINS'S TEN ATES ARE BIT-IDENTICAL TO ITS OWN PRE-CHANGE OUTPUT. ROWS WERE MEASURED SEQUENTIALLY AT THEIR LANDING POINT, SO EFFECTS ARE NOT STRICTLY ADDITIVE.

#	Change	Effect
Solver restructures (numerically equivalent or accuracy-gated arithmetic)		
2	Exact early-accept for the χ^2 gate ($S \succeq \sigma^2 I$ bound)	MSCKF χ^2: 0.273→0.079 ms
3	Square-root-form EKF update (Sec. III-C)	S+inv: 0.436→0.263 ms; K: 0.272→0.160 ms
4	$M_a = PH^\top$ as one GEMM instead of per-block products	0.668→0.461 ms
7	Measurement compression + χ^2 restructuring	MSCKF stage: 1.535→1.000 ms
Storage and layout changes		
1	Feature DB: separate key order from payload storage	9,556 memcpy/frame → 0
5	`STATE_COV_CAPACITY` 512→384 (matches true worst case, 341)	Cov update: 0.745→0.506 ms
6	DHAT-guided allocation cuts (3 sites)	1,322,658→825,790 blocks
Mixed (arithmetic and data placement restructured together)		
8	M_a/covariance-update restructuring (first pass)	Per-frame: 7.40→5.43 ms

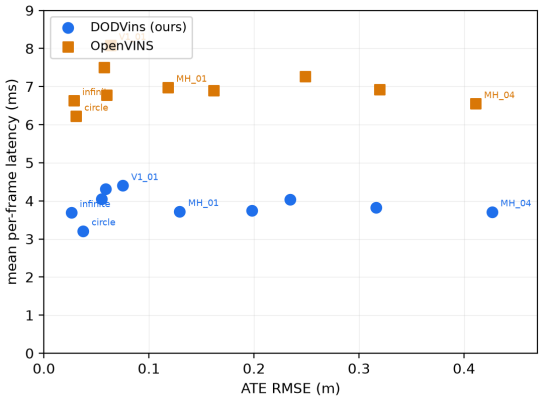

Fig. 1. The accuracy/efficiency trade-off per sequence, x86: mean per-frame latency (y) against ATE (x). DODVins's points sit below OpenVINS's on every sequence at comparable accuracy; the speedup does not purchase accuracy anywhere on the benchmark.

M_a accumulation loop was already structured identically to OpenVINS's — though restructuring it was still worth 2.2× for a different, profiled reason (row 4).

The classification above is the paper's answer to which kind of change buys what. The cleanly attributable solver restructures account for 1.22 ms/frame of stage-level savings; the attributable storage changes account for 0.24 ms/frame plus the feature-database memcpy elimination; row 8 restructured arithmetic and data placement together and cannot be attributed to one class. The tail-latency signature of Section V-D — the p99 advantage exceeding the mean advantage — is the evidence that data placement contributes beyond its isolated arithmetic: allocator activity and cache misses land in spikes, and that is where DODVins pulls furthest ahead. On this codebase, plausible causal narratives about where time goes have been wrong more often than right; every change in Table V was accepted only after a profile, not a guess.

F. Relative pose error: locating the accuracy gap

Aggregate ATE does not indicate whether initialization or the filter's steady-state drift rate is responsible for a sequence's result. We compute relative pose error (RPE) over a 1-second sliding window — alignment-free, so it measures local drift rate independent of any accumulated global offset. Table VI shows DODVins's steady-state drift rate is better than OpenVINS's on five of eight EuRoC sequences, comparable on two (MH_01, MH_05; within 2%), and worse on one (MH_02, by 3.5%). The filter itself is not, in aggregate, the source of the ATE gap in Table II.

TABLE VI

RELATIVE POSE ERROR, 1 S WINDOW (M), ALIGNMENT-FREE.

Sequence	DODVins	OpenVINS
MH_01	0.0407	0.0406
MH_02	0.0384	0.0371
MH_03	**0.0853**	0.0936
MH_04	**0.0811**	0.0913
MH_05	0.1066	0.1078
V1_01	**0.0084**	0.0123
V1_02	**0.0093**	0.0140
V1_03	**0.0140**	0.0222

Decomposing MH_01 and MH_02 by elapsed time confirms where the gap is: DODVins initializes 1.8–2.8 s *earlier* and starts from a worse initial state (first-5 s RMSE on MH_02: 0.2133 m vs. 0.1188 m) despite near-identical drift thereafter — MH_02's overall ATE gap is an initialization transient, not a difference in filter quality.

G. Fixing what silent failure paths hid

The measurement effort in Sections V-F–VI surfaced four defects on the frame-input path that a purely accuracy-driven evaluation would not have found: none fired on the ten benchmarked sequences, and none moved a single ATE digit when fixed. An IMU ring buffer discarded the *newest* sample once full, so a sequence whose initializer had not fired within 10,000 samples (≈50 s at 200 Hz) would freeze permanently, with no counter to make the failure visible. An IMU-window selection routine silently truncated at a scratch-buffer capacity and returned success. A propagation step's boolean failure return was discarded by its caller, so a non-advancing timestamp (reachable on a live sensor feed, never

TABLE VII
PER-FRAME PIPELINE LATENCY ON THE JETSON ORIN NANO SUPER (TRACKING + ESTIMATION, EXCLUDING BAG I/O; MS; SAME TOOLCHAIN CONTAINER ON BOTH SIDES, ONLY THE ISA DIFFERS).

	DODVins		OpenVINS	
Sequence	p50	p99	p50	p99
MH_01	27.3	51.4	43.9	73.3
MH_02	27.3	51.1	42.9	71.1
MH_03	30.1	58.6	39.0	72.5
MH_04	27.0	54.4	41.8	74.2
MH_05	28.9	53.9	38.2	68.2
V1_01	35.3	57.1	47.4	81.8
V1_02	33.0	63.4	42.4	70.5
V1_03	30.6	62.7	38.8	68.0
circle	28.6	50.9	34.5	56.5
infinite	29.3	45.0	34.1	54.8

in bag playback) would still trigger a full update against an unpropagated state. All three now count their own rejections, all ten ATEs remained bit-identical, and every counter reads zero on every benchmarked sequence, confirming these paths were genuinely unreached rather than merely undetected. The methodological point matters as much as the fixes: an accuracy benchmark that only exercises well-behaved input data will not find defects that only manifest under input it does not produce.

VI. EMBEDDED RESULTS ON JETSON ORIN NANO AND A CROSS-PLATFORM DEFECT

To test the design where memory layout is supposed to matter most, both estimators were built and run on an NVIDIA Jetson Orin Nano Super (6-core Cortex-A78AE, 8 GB LPDDR5, JetPack 6 / L4T R36.4) inside an aarch64 Ubuntu 20.04 container with GCC 9.4.0 and OpenCV 4.2.0 — the identical toolchain as the x86 container of Section IV, so the only variable between platforms is the processor. All ten sequences ran on-device for both estimators through the same direct-rosbag transport as x86, with per-frame timing and peak RSS recorded as on x86.

Latency. Table VII shows DODVins is faster on all ten sequences at both the median (1.16–1.61×) and the 99th percentile (1.08–1.43×). Two honest observations qualify this. First, the margin is *narrower* than on the desktop x86, where the median advantage was 1.63–1.85×: whatever the mechanism behind the desktop tail advantage (Section V), it does not grow on the small core — p99 improves by less than p50 on 8 of 10 sequences here. Second, absolute latencies are large: at EuRoC's 20 Hz rate (50 ms budget) OpenVINS's p99 exceeds the frame budget on every sequence, and the DODVins's does on nine of ten, by 1–13 ms. Neither estimator is comfortably real-time on this device at 20 Hz as configured; the DODVins is the only one whose *median* stays within budget on all ten sequences, and its peak RSS is flat (184–199 MB across all ten sequences, against OpenVINS's 98–179 MB variable). The bounded-memory design's cost is visible here too: it pre-allocates that working set whether or not a sequence needs it. Per-sequence latency distributions for both estimators show the same direction; the margin does not transfer in full.

Accuracy on-device. DODVins completed all ten sequences without divergence. ATEs (the seven sequences for which the bag transport yields ground truth on this device) sit in the same family as the x86 container's: MH_01 0.1316 vs. 0.1293, MH_02 0.1928 vs. 0.2080, MH_03 0.2473 vs. 0.2343, MH_04 0.3505 vs. 0.4267, MH_05 0.3358 vs. 0.3285, KAIST circle 0.0835 vs. 0.0374, infinity 0.0365 vs. 0.0261. Per Section IV-C's lesson these are quoted as their own column, not compared for parity: the KLT frontend's rounding behavior is part of the platform, and KAIST circle — the initialization-transient-sensitive sequence of Section V-F — moves the most.

What was not validated. End-to-end wall-clock via `hyperfine`, thermal/frequency state during measurement, and power draw were not controlled or recorded; the per-frame latency figures above are the embedded comparison this paper makes, and the CPU frequency governor was left at its default on a passively cooled module, which a desktop measurement does not have to worry about and this one does.

The defect found. A unit test covering a fallback path of the dynamic initializer (Section III-D's primary initializer's own fallback-of-a-fallback, reachable only when the feature-less initializer itself fails) passed on x86 and failed on the Jetson. The failure initially appeared to be a wrong-root selection in a 6×6 companion-matrix eigenvalue solve — the recovered gravity direction printed as 171° from vertical, which looks like a sign error. Instrumenting every candidate root on both platforms showed this was a misdiagnosis: both platforms select the identical root by identical cost criteria, and OpenVINS's own printed diagnostic for that same quantity reads the same ≈171° on a passing x86 run, because that print was never part of the root-selection or acceptance decision in the first place (its associated threshold defaults to 10^9 and is never overridden by any shipped configuration). The actual divergence was a factor of three in a downstream $|g|$-convergence residual (x86: 0.00034; aarch64: 0.00108), straddling a hard-coded 10^{-3} absolute acceptance tolerance — a platform-dependent rounding difference through a QR-iteration eigenvalue solve (SSE2 vs. NEON reduction order), not an algorithmic error. The tolerance was widened to 3×10^{-3}, with the measured spread that justifies the new value recorded alongside it; the fix was verified not to move the eigenvalue selected, the $|g|$ residual's sign, or the ten sequences' ATEs on x86 (this path is unreached on all ten). We report it even though it affected a fallback of a fallback that Table II's benchmark never exercises, because it is exactly the kind of defect a same-platform test suite cannot find, and one we would not have found without attempting the embedded deployment this section reports.

VII. INDEPENDENT EVALUATION ON TUM VI

Every result above was measured on datasets the pipeline's constants were tuned against, and our own altitude work showed that inherited constants are the first thing that breaks on new data. As an independent test, the pipeline was run on the TUM VI benchmark [7] with *only its calibration adapted*:

TABLE VIII
TUM VI ROOM SEQUENCES, ATE RMSE (M). BOTH ESTIMATORS RAN WITH THE DATASET'S OWN KALIBR CALIBRATION; OPENVINS RECEIVED THE SAME RELEASE AS ITS CONFIG, SO THE COMPARISON IS CALIBRATION-FOR-CALIBRATION.

Sequence	DODVins	OpenVINS
room1	0.0841	**0.0693**
room2	0.0652	**0.0542**
room3	**0.0661**	0.0871
room4	**0.0276**	0.0359
room5	**0.0910**	0.1011
room6	0.0363	**0.0327**
Mean	0.0617	0.0634

TABLE IX
FGI MASALA, ATE (M) AT 4 M/S. × MARKS DIVERGENCE. NO SHIPPED CONFIGURATION COVERS THE ALTITUDE ENVELOPE; A SINGLE INHERITED CONSTANT SEPARATES DODVINS FROM COVERING IT.

Altitude	DODVins (σ=1)	DODVins (tuned σ)	OpenVINS
40 m	12.81	12.81	**4.19**
60 m	1367.7 ×	48.25	**12.22**
80 m	56303 ×	**23.17**	145059 ×
100 m	94897 ×	110.27	**57.21**

the dataset's own pinhole-equi-512 Kalibr release read verbatim (Kannala-Brandt 4 through the existing equidistant model, $T_{\text{cam\_imu}}$ in the KAIST convention, BMI055 noise densities as shipped), with every estimator knob the marked EuRoC placeholder. This is also the first fisheye camera the pipeline has processed.

Table VIII and Figure 2 show the outcome: all six room sequences initialize and converge with zero retuning, at accuracy parity with OpenVINS (better on 3 of 6, mean 0.062 m against 0.063 m), and with comparable drift: the 1 s-window relative pose error (Section V-F's metric) is 0.0064–0.0101 m for the DODVins against 0.0053–0.0101 m for OpenVINS across the six sequences. For a pipeline whose every constant was inherited from EuRoC and KAIST, converging untuned on a third dataset family (different camera model, different IMU, different scene scale) is the strongest available evidence that the estimator is not overfit to its benchmarks.

VIII. HIGH-ALTITUDE OUTDOOR EVALUATION

EuRoC, TUM VI, and KAIST are indoor benchmarks with scene depth of a few meters. Outdoor UAV flight is the harder regime for stereo VIO: the FGI Masala dataset [8] flies a downward-facing 30 cm-baseline stereo rig at 40/60/80/100 m of altitude, where the depth-to-baseline ratio reaches 130–330 and stereo depth error grows with its square. Both estimators were run on the four 4 m/s sequences with the dataset's own calibration, and DODVins in two configurations: with the pixel measurement noise inherited from EuRoC (σ=1), and with a single per-altitude value fitted on these sequences (σ=10–15; reported for completeness — it is tuned on the test data, not a tuning-free claim).

Table IX makes three points. First, no shipped configuration covers the envelope: DODVins with the inherited constant diverges above 40 m, and OpenVINS's default configuration — which initializes at all four altitudes — diverges at 80 m (490 km of estimated path over a 1.2 km trajectory). Second, the dominant lever is a single inherited measurement-noise constant, the same species of failure as Section IV-C but at outdoor scale: correcting it recovers DODVins across 60–100 m. Third, neither estimator is uniformly better where both run — OpenVINS wins at 40/60/100 m, DODVins is the only survivor at 80 m. Per-frame cost is unchanged outdoors (DODVins median 2.6–3.9 ms, p99 6.8–8.2 ms, far inside the 16 Hz budget), so the speed claim of Section V-D is not indoor-specific; what the regime breaks is measurement weighting for long-baseline tracks, in both pipelines. The diagnosis came from DODVins's per-path counters rather than from accuracy alone: at 80 m the conditioning gate rejects most triangulations and the filter starves of features before it ever mis-weights one.

IX. LIMITATIONS

This is an implementation and measurement study, not a new estimator. Every filter equation in the primary pipeline is OpenVINS's; DODVins's contribution is memory layout, and the honest framing of every result in this paper follows from that: layout changes speed, not accuracy, and Table II bears that out.

Accuracy is parity, not superiority. Better on 4 of 10 sequences, mean ATE 0.156 m against OpenVINS's 0.158 m. A reader looking for a strict accuracy win should not find one here; we report Table II in full rather than a favorably selected subset.

SchurVINS support is built but not benchmarked against the authors' fork. DODVins includes a data-oriented SchurVINS update path (Section III-A's constraints applied to [3]'s algorithm), gated behind a runtime flag that defaults off, and verified to run the full MH_01 sequence to completion when enabled. No result in this paper uses it: an earlier speed comparison against the authors' own released `ov_SchurVINS` fork was withdrawn because the fork as released aborts on MH_01 in our environment (in both its serial and subscribe modes) and the build that produced the earlier comparison no longer exists. We do not quote a number we cannot reproduce.

Embedded comparison covers latency and memory, not the wall. Section VI reports both estimators on all ten sequences on the Orin Nano with per-frame timing and peak RSS, but end-to-end wall-clock under `hyperfine`, thermal state, and CPU frequency governor behavior were not controlled or recorded; on a passively cooled module these can move both sides. The allocation profile (DHAT) is x86-only.

The TUM VI evaluation is x86-only and deliberately untuned. Section VII runs on the desktop toolchain; repeating it on the Orin Nano is future work. No estimator knob was swept on TUM VI — that is the design of the test, and it means the numbers there are the untuned configuration's, not the pipeline's best.

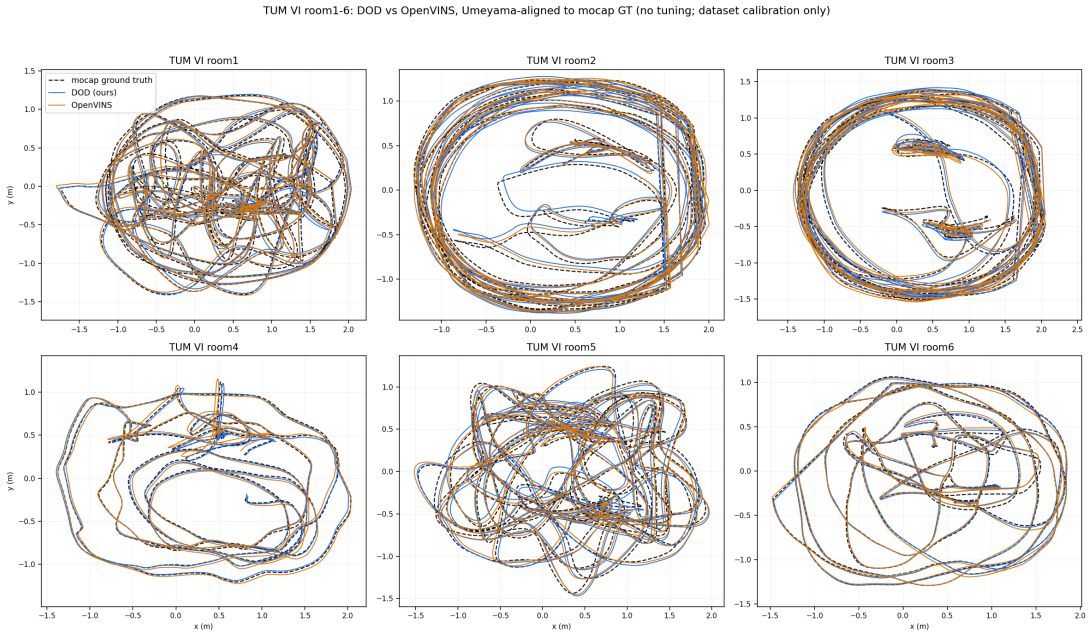

Fig. 2. TUM VI room1–6: both estimators Umeyama-aligned to the mocap ground truth. All twelve runs (six per estimator) initialize and converge; no sequence was tuned for.

Monocular operation is not supported. The tracker front-end requires exactly two cameras; a single-camera front end would need new tracking code (the estimator core already parameterizes camera count and would not require changes) and has not been attempted.

KAIST initialization latency was investigated and the obvious fix rejected on evidence, not assumed away. On KAIST circle, the primary initializer requires an accelerometer-variance jerk that does not occur until 25.4 s into the sequence (OpenVINS initializes at 3.7 s via a different trigger: its own KAIST configuration enables its zero-velocity update at start, which changes which initializer branch it takes). Reconfiguring DODVins to match — initializing on the first stationary window instead of waiting for a jerk — was implemented, measured, and found to produce a *worse* trajectory (comparing only the time window both configurations cover, to rule out a scoring artifact: 0.0464 m vs. the shipped 0.0375 m on circle, 0.0363 m vs. 0.0261 m on infinity, both roughly 24% worse), and was reverted. We report the negative result rather than omit it: the correct fix requires replicating OpenVINS's paired zero-velocity-update configuration for this specific sequence, which remains unimplemented.

X. Conclusion

Reimplementing OpenVINS's MSCKF pipeline under strict data-oriented constraints — fixed-capacity per-frame state, no virtual dispatch — yields a consistent 1.76–2.10× end-to-end wall-clock speedup on ten public sequences at accuracy parity (mean ATE 0.156 m vs. 0.158 m; better on 4 of 10), with steady-state drift comparable or better on 7 of 8 EuRoC sequences — the aggregate gap is an initialization transient, not filter quality — and every optimization accepted only after leaving all ten trajectories bit-identical to their pre-change output. The embedded deployment (Section VI) shows the advantage transfers in direction but not in size (faster on all ten sequences on the Orin Nano, by a narrower margin than on desktop x86) and surfaced a genuine cross-platform floating-point defect in an otherwise-unreached code path, illustrating that even a same-architecture, bit-identical-trajectory validation regime does not by itself certify cross-platform correctness. Finally, an untuned run on the TUM VI benchmark (Section VII), a third dataset family and the first fisheye camera model, converges on all six room sequences at parity with OpenVINS, evidence that the estimator is not overfit to the datasets its constants were tuned on.

For a practitioner the guidance is narrow but concrete: on desktop x86, DODVins delivers the same MSCKF trajectories as OpenVINS for roughly half the wall-clock and a quarter to a third less processor time, with a frame-latency tail inside the sensor's budget and a bounded working set — at the price of higher average memory utilization and no monocular support. On a Jetson-class SoC every statement keeps its direction; only the margin narrows. Teams whose estimator shares the compute budget with control and navigation get the freed share without changing a filter equation; teams relying on loop closure or tight memory ceilings should weigh the trade-offs in Section efsec:limitations. The claim is about implementation, not estimator novelty — layout plus measured solver restructures — and the measurements support both attributions.

REFERENCES

[1] P. Geneva, K. Eckenhoff, W. Lee, Y. Yang, and G. Huang, "OpenVINS: A Research Platform for Visual-Inertial Estimation," *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, Paris, France, 2020.

[2] A. I. Mourikis and S. I. Roumeliotis, "A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation," *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, Rome, Italy, 2007, pp. 3565–3572.

[3] Y. Fan, T. Zhao, and G. Wang, "SchurVINS: Schur Complement-Based Lightweight Visual Inertial Navigation System," *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024. arXiv:2312.01616.

[4] C. Peng *et al.*, "sqrtVINS: Square-Root Filtering for Visual-Inertial Navigation Systems," *IEEE Trans. Robotics (T-RO)*, 2025.

[5] D. G. Kottas, K. J. Wu, and S. I. Roumeliotis, "Detecting and Dealing with Hovering Maneuvers in Vision-Aided Inertial Navigation Systems," *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Tokyo, Japan, 2013, pp. 3172–3179.

[6] R. Fabian, *Data-Oriented Design*, self-published, 2018. https://www.dataorienteddesign.com/dodmain/

[7] D. Schubert, T. Goll, N. Demmel, V. Usenko, J. Stückler, and D. Cremers, "The TUM VI Benchmark for Evaluating Visual-Inertial Odometry," *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Madrid, Spain, 2018.

[8] A. George, N. Koivumäki, T. Hakala, J. Suomalainen, and E. Honkavaara, "Visual-Inertial Odometry Using High Flying Altitude Drone Datasets," *Drones*, vol. 7, no. 1, p. 36, 2023.